Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 3
«Розроблення підсистеми ідентифікації та автентифікації»
з дисципліни «Безпека програмного забезпечення»

Варіант 12

Виконав: здобувач групи ІП-33
Ломакін Максим

Перевірив:
Курченко Олег Анастасійович

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи – набути практичних навичок проєктування та реалізації підсистеми ідентифікації й автентифікації користувача: безпечного зберігання паролів, кількісного обґрунтування їхньої стійкості, захисту від автоматизованого підбору та застосування другого чинника.

Завдання: розрахувати мінімальну довжину пароля за формулою Андерсона з параметрами варіанта; реалізувати зберігання та перевірку пароля за функцією формування ключа варіанта із сіллю, перцем і порівнянням за сталий час; порівняти захищене зберігання з вразливим (SHA-256 без солі); реалізувати блокування облікового запису та другий чинник автентифікації за варіантом; розробити набір автоматизованих тестів.

Варіант 12. Функція формування ключа – PBKDF2-HMAC-SHA256; другий чинник – одноразові коди відновлення; схема блокування – фіксоване блокування на 15 хв після 5 невдалих спроб; параметри формули Андерсона: період дії пароля T = 30 діб, швидкість спроб R = 20 за хвилину, допустима ймовірність підбору $P = 10^{-6}$. Вивід команди `python3 -m common.variants 12 --lab 3`:

```
Варіант 12

  LAB3
    kdf: PBKDF2-HMAC-SHA256
    second_factor: одноразові коди відновлення
    lockout: фіксоване блокування на 15 хв після 5 невдалих спроб
    anderson: period_days=30, guesses_per_minute=20, max_probability=1e-06
```

2 ТЕОРЕТИЧНІ ВІДОМОСТІ

Підсистема автентифікації має витримувати дві різні атаки на пароль. Онлайн-підбір виконується через штатну процедуру входу, тому його швидкість R обмежує сама система. Для нього формула Андерсона [3] дає нижню межу простору паролів $G \geq R \cdot T / P$ і мінімальну довжину $L = \lceil \log G / \log A \rceil$; з неї в роботі виведено вимогу до довжини пароля під час реєстрації. Офлайн-підбір починається після викрадення бази облікових записів: порушник перевіряє кандидати на власному обладнанні, і стримує його лише вартість одного обчислення функції зберігання [1].

Тому пароль зберігається як результат функції формування ключа. Варіант задає PBKDF2-HMAC-SHA256 [5]: ключ отримують багаторазовим застосуванням HMAC-SHA256 до пароля й солі, а кількість ітерацій є єдиним параметром вартості. Обрано 600 000 ітерацій – мінімум, рекомендований для цієї функції [4]. PBKDF2 не потребує великого обсягу пам'яті, тому на графічних процесорах прискорюється краще, ніж Argon2id; це обмеження варіанта компенсується кількістю ітерацій і перцем.

Сіль – випадкове значення, окреме для кожного пароля, яке зберігається поряд із ключем. NIST вимагає солі не коротшої за 32 біти [2]; у роботі взято 16 байтів (128 біт), як вимагає завдання. Перець – секрет, спільний для всіх паролів, який зберігається поза базою. У каркасі він застосовується як HMAC-SHA256 над паролем із ключем зі змінної середовища `AUTH_PEPPER`, і результат подається на вхід PBKDF2. Сіль і перець закривають різні загрози, тому використано обидва.

Порівняння обчисленого ключа зі збереженим звичайним оператором рівності завершується на першому відмінному байті, і час відповіді залежить від довжини спільного префікса. Щоб не створювати такого каналу витоку, ключі паролів і хеш-суми кодів відновлення порівнюються функцією `hmac.compare_digest`, час роботи якої не залежить від місця розбіжності [6].

Швидкість R у формулі Андерсона обмежує блокування облікового запису. Варіант задає фіксоване блокування на 15 хв після 5 невдалих спроб. Другий чинник варіанта – одноразові коди відновлення: заздалегідь згенеровані випадкові рядки, кожен з яких дійсний один раз [2]. Оскільки код є випадковим значенням високої ентропії, а не вигаданим людиною паролем, для його зберігання достатньо швидкої хеш-функції: перебір 2^{60} варіантів неможливий і без уповільнення.

3 ОПИС РОЗРОБЛЕНОГО РІШЕННЯ

3.1 Структурна схема підсистеми

Підсистему побудовано з незалежних модулів пакета `auth`, які об'єднує клас `AuthService` (рисунок 3.1). Під час входу він спершу запитує в `FixedLockout`, чи дозволено спробу, потім перевіряє пароль функцією `verify_password`, далі – код відновлення методом `RecoveryCodes.verify`, і повідомляє результат спроби механізмові блокування. Під час реєстрації перевіряється мінімальна довжина пароля, обчислена за формулою Андерсона, і видаються 10 кодів відновлення. Перець береться зі змінної середовища і в сховище облікових записів не потрапляє.

Модуль `auth/anderson.py` узято з репозиторію курсу [8] без змін, у каркасі `auth/passwords.py` реалізовано дві функції. Склад програмного коду наведено в таблиці 3.1. Використано лише стандартну бібліотеку Python 3.12 (`hashlib`, `hmac`, `secrets`, `os`) і pytest 8.3.3.

Таблиця 3.1 – Модулі підсистеми

Файл	Вміст
`auth/anderson.py`	формула Андерсона: `required_space`, `min_length`, `entropy_bits` (каркас курсу)
`auth/passwords.py`	`hash_password`, `verify_password` за PBKDF2-HMAC-SHA256; вразливий зразок `hash_password_insecure` і перець `_peppered` (каркас курсу)

Файл	Вміст
`auth/lockout.py`	клас `FixedLockout` і функції `record_attempt`, `is_allowed`
`auth/recovery.py`	`generate_codes` і клас `RecoveryCodes`
`auth/service.py`	клас `AuthService`: реєстрація та вхід
`demo.py`	демонстрація всіх кроків роботи
`tests/test_variant.py`	34 автоматизовані тести

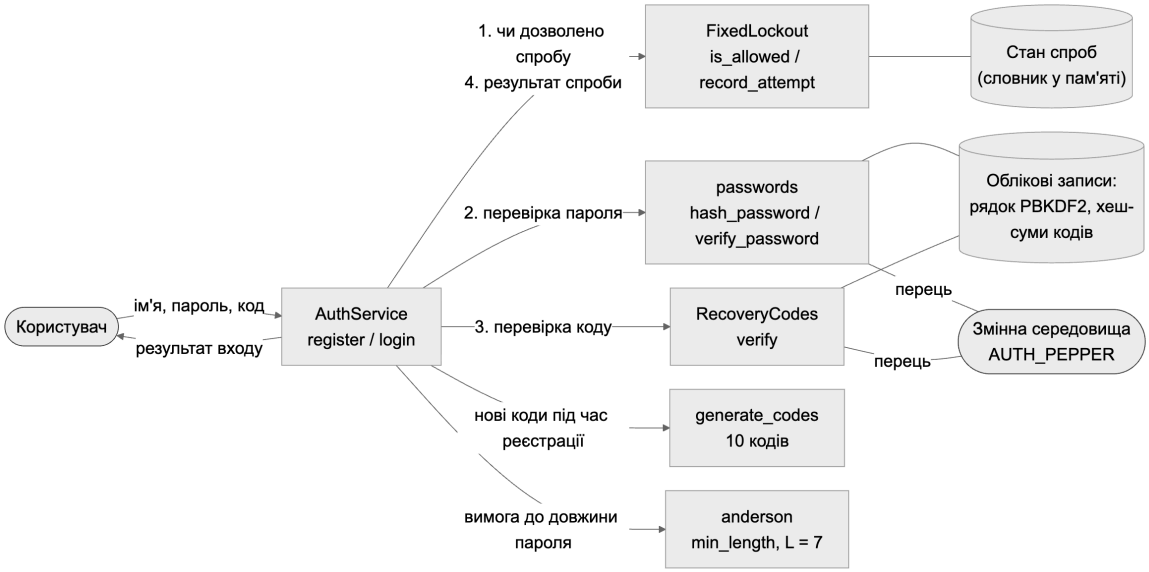


Рисунок 3.1 – Структурна схема підсистеми автентифікації

3.2 Зберігання та перевірка пароля

Збережений рядок складається з чотирьох полів, розділених знаком `$`: назва алгоритму `pbkdf2_sha256`, кількість ітерацій, сіль і ключ у кодуванні Base64. Параметри зберігаються разом із ключем, тому кількість ітерацій можна збільшити пізніше, не втративши можливості перевіряти старі записи. Параметри: 600 000 ітерацій, сіль 16 байтів з `os.urandom`, ключ 32 байти (розмір виходу SHA-256; довший ключ вимагав би другого проходу PBKDF2 без додаткової стійкості).

```
def hash_password(password: str) -> str:
    """БЕЗПЕЧНО: PBKDF2-HMAC-SHA256 із сіллю та «перцем».

    Формат рядка: pbkdf2_sha256$<ітерації>$<b64(сіль)>$<b64(ключ)>.
    """
    salt = os.urandom(SALT_BYTES)
    key = _derive(password, salt, ITERATIONS)
    return f"{ALGORITHM}${ITERATIONS}${_b64(salt)}${_b64(key)}"

def verify_password(stored: str, password: str) -> bool:
    """Перевірка пароля. Порівняння ключів — лише за сталий час."""
    try:
        algorithm, iterations, salt, expected = stored.split("$")
        if algorithm != ALGORITHM:
            return False
        key = _derive(password, _unb64(salt), int(iterations))
        return hmac.compare_digest(key, _unb64(expected))
    except ValueError:
        return False
```

```
def _derive(password: str, salt: bytes, iterations: int) -> bytes:
    data = _peppered(password.encode())
    return hashlib.pbkdf2_hmac("sha256", data, salt, iterations, KEY_BYTES)

def _peppered(data: bytes) -> bytes:
    return hmac.new(PEPPER, data, hashlib.sha256).digest() if PEPPER else data
```

Функція `_derive` спершу застосовує перець: пароль замінюється значенням HMAC-SHA256 з ключем `AUTH_PEPPER`, і вже воно разом із сіллю подається в PBKDF2. Сіль унікальна для запису й зберігається в рядку, перець спільний і зберігається поза ним. `verify_password` розбирає рядок, відновлює сіль і кількість ітерацій, обчислює ключ для пароля-кандидата і порівнює його зі збереженим функцією `hmac.compare_digest`. Рядок з іншим алгоритмом, нечисловою кількістю ітерацій або пошкодженим Base64 дає `False`, а не виняток: помилка розбору не повинна відрізнятися для зовнішнього спостерігача від хибного пароля.

3.3 Блокування облікового запису

Клас `FixedLockout` зберігає для кожного облікового запису кількість невдалих спроб поспіль і момент, до якого діє блокування. Стан міститься в словнику в пам'яті процесу; час передається параметром `now`, тому поведінка відтворювана й перевіряється без очікування.

```
def is_allowed(self, account: str, now: float) -> bool:
    state = self._accounts.get(account)
    return state is None or now >= state.locked_until

def record_attempt(self, account: str, ok: bool, now: float) -> None:
    if not self.is_allowed(account, now):
        return
    state = self._accounts.get(account)
    if ok or (state and state.failures >= self.max_failures):
        self._accounts.pop(account, None)
        state = None
    if ok:
        return
    if state is None:
        state = self._accounts[account] = _State()
    state.failures += 1
    if state.failures >= self.max_failures:
        state.locked_until = now + self.lock_seconds
```

Прийнято такі рішення. П'ята невдала спроба встановлює `locked_until = now + 900`; спроба дозволена, коли `now >= locked_until`, тобто блокування триває рівно 15 хв. Спроби під час блокування не враховуються й не подовжують його – інакше порушник міг би тримати обліковий запис заблокованим необмежено довго одним запитом на 15 хв. Після спливання інтервалу відлік починається з нуля. Успішний вхід скидає лічильник. Функції рівня модуля `record_attempt` і `is_allowed` працюють зі спільним екземпляром із параметрами варіанта.

3.4 Одноразові коди відновлення

Код складається з 12 символів алфавіту з 32 знаків (латинські літери й цифри без схожих на вигляд 0, O, 1, I), згенерованих модулем `secrets`, і записується трьома групами по чотири символи. Ентропія коду – $12 \cdot \log_2 32 = 60$ біт. Клас `RecoveryCodes` зберігає не коди, а їхні хеш-суми SHA-256, обчислені з перцем: витік сховища не розкриває дійсних кодів.

```
def generate_codes(count: int = CODE_COUNT) -> list[str]:
    """Повертає count різних кодів виду XXXX-XXXX-XXXX (60 біт ентропії)."""
    codes: dict[str, None] = {}
    while len(codes) < count:
        raw = "".join(secrets.choice(ALPHABET) for _ in range(CODE_LENGTH))
        parts = (raw[i:i + GROUP] for i in range(0, CODE_LENGTH, GROUP))
        codes["-".join(parts)] = None
    return list(codes)

def _digest(code: str) -> bytes:
    normalized = "".join(ch for ch in code.upper() if ch.isalnum())
    return hashlib.sha256(_peppered(normalized.encode())).digest()

def verify(self, code: str) -> bool:
    """Приймає кожен код один раз; порівняння — за сталий час."""
    candidate = _digest(code)
    found = -1
    for index, digest in enumerate(self._digests):
        if hmac.compare_digest(digest, candidate):
            found = index
    if found < 0:
        return False
    del self._digests[found]
    return True
```

Перед обчисленням хеш-суми введення нормалізується: відкидаються дефіси й пробіли, літери переводяться у верхній регістр. `verify` порівнює хеш-суму кандидата з кожною збереженою функцією `hmac.compare_digest` і не перериває цикл після збігу. Знайдена хеш-сума вилучається зі списку, тому повторне пред'явлення того самого коду відхиляється.

3.5 Процедура входу

```
def login(self, account: str, password: str, code: str,
          now: float) -> str:
    """Перевіряє пароль, а потім код; обидві відмови рахує блокування."""
    if not self.lockout.is_allowed(account, now):
        return LOCKED
    user = self._users.get(account)
    stored = user.password_hash if user else self._dummy_hash
    ok = verify_password(stored, password) and user is not None
    ok = ok and user.codes.verify(code)
    self.lockout.record_attempt(account, ok, now)
    return OK if ok else DENIED
```

Заблокований обліковий запис отримує відмову до будь-яких обчислень. Код відновлення перевіряється лише після вірного пароля, щоб хибний пароль не витрачав код. Невдача на будь-якому з двох етапів реєструється як невдала спроба, тому перебір

кодів обмежено так само, як перебір паролів. Для неіснуючого облікового запису обчислюється PBKDF2 над службовим рядком: час відповіді не показує, чи зареєстровано ім'я. Зовнішня відповідь не розрізняє хибний пароль і хибний код.

4 РЕЗУЛЬТАТИ

4.1 Розрахунок за формулою Андерсона

Контрольний приклад лекції ($R = 10$, $T = 90$ діб, $P = 10^{-6}$, $A = 95$) відтворюється: команда `python -m auth.anderson` виводить «G ≥ 1.296e+12; мінімальна довжина L = 7; ентропія 46.0 біт».

Для варіанта 12: $T = 30 \cdot 24 \cdot 60 = 43\,200$ хв; кількість спроб за період $R \cdot T = 20 \cdot 43\,200 = 864\,000$; нижня межа простору паролів $G \geq 864\,000 / 10^{-6} = 8{,}64 \cdot 10^{11}$; $\ln G = 27{,}485$. Довжину обчислено як $L = \lceil \ln G / \ln A \rceil$, ентропію – як $H = L \cdot \log_2 A$. Наприклад, для $A = 95$: $27{,}485 / 4{,}554 = 6{,}035$, $L = 7$, $H = 7 \cdot 6{,}570 = 46{,}0$ біта. Результати наведено в таблиці 4.1; останній стовпець – фактична ймовірність підбору $R \cdot T / A^L$, яка в усіх рядках не перевищує 10^{-6}.

Таблиця 4.1 – Мінімальна довжина та ентропія пароля для параметрів варіанта

A	Алфавіт	ln A	ln G / ln A	L	A^L	H, біт	$R \cdot T / A^L$
26	малі латинські літери	3,258	8,436	9	$5{,}43 \cdot 10^{12}$	42,3	$1{,}6 \cdot 10^{-7}$
62	літери обох регістрів і цифри	4,127	6,660	7	$3{,}52 \cdot 10^{12}$	41,7	$2{,}5 \cdot 10^{-7}$
95	усі друковані символи ASCII	4,554	6,035	7	$6{,}98 \cdot 10^{13}$	46,0	$1{,}2 \cdot 10^{-8}$

Вивід програми (останній рядок – розрахунок для швидкості, обмеженої блокуванням, розділ 5):

```
T = 30 діб = 43200 хв; R·T = 864000
G ≥ 8.640e+11
A    log G / log A    L    H, біт
26   8.436            9    42.3
62   6.660            7    41.7
95   6.035            7    46.0
з блокуванням: R = 0.333/хв, G ≥ 1.440e+10, L = 8 (A=26), 6 (A=62), 6 (A=95)
```

4.2 Зберігання пароля та порівняння з вразливим зразком

Вхідні дані – синтетичні паролі `correct horse`, `same`, `password123` і тестовий перець `lab3-synthetic-pepper`. Самоперевірка кроку 3: вірний пароль приймається, хибний відхиляється, два рядки для одного пароля різні. Додатково перевірено, що зі зміненим перцем вірний пароль не проходить.

```
збережений рядок:
pbkdf2_sha256$600000$BLC5Ve6p2jq5SdL5rH6RrA==$7qnQYd1TAw4wbpi8WM36K8fYD4wc8O2di6Rzku4M+
XY=
алгоритм: pbkdf2_sha256; ітерацій: 600000; сіль: 16 байтів; ключ: 32 байтів
вірний пароль: True
хибний пароль: False
солі різні: True
вірний пароль з іншим перцем: False
```

Порівняння кроку 4: SHA-256 без солі двічі дає ту саму хеш-суму, PBKDF2 із сіллю – два різні рядки.

```
SHA-256, спроба 1: ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
SHA-256, спроба 2: ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
вразливі хеші однакові: True
PBKDF2, спроба 1:
pbkdf2_sha256$600000$BCoyjpxalPGQGvrlLdevbw==$km1b3iN+cbJboG/26aq5ykw0mcXi15ZyvQMOGrj2F
1A=
PBKDF2, спроба 2:
pbkdf2_sha256$600000$CAuiuP8D5ysEYmNOuVXisg==$PAhw48z+nwdKrgeaoBitfnuwxe8UvrwFFLH1CpQ/L
Ok=
захищені рядки різні: True
```

Час обчислення виміряно на стенді (Python 3.12, один потік):

```
hash_password (медіана з 7): 126.7 мс
hash_password_insecure (середнє з 200000): 0.457 мкс
відношення: 277104
паролів за секунду: PBKDF2 7.9, SHA-256 2186642
```

4.3 Сценарій блокування

Обліковий запис alice отримує п'ять невдалих спроб з інтервалом 10 с (час наведено у форматі хв:с). Після четвертої спроби вхід ще дозволено, після п'ятої – ні. Блокування, встановлене в момент 00:40, діє до 15:40: за секунду до цього `is_allowed` повертає `False`, рівно в 15:40 – `True`. Наступна невдала спроба рахується як перша, успішна – скидає лічильник. Обліковий запис bob після чотирьох невдалих спроб не блокується.

```
час   спроба       невдалих  дозволено далі
00:00 невдала      1         True
00:10 невдала      2         True
00:20 невдала      3         True
00:30 невдала      4         True
00:40 невдала      5         False
блокування до 15:40
15:39 is_allowed = False
15:40 is_allowed = True
15:41 is_allowed = True
15:40 невдала      1         True
15:45 успішна      0         True
bob: 4 невдалі -> is_allowed = True
bob: успішна -> невдалих 0
```

4.4 Коди відновлення

Згенеровано 10 кодів (синтетичні, дійсні лише в межах одного запуску). Третій код приймається один раз, після чого дійсних кодів стає 9; повторне пред'явлення

відхиляється. Код, уведений малими літерами без дефісів, приймається; вигаданий код відхиляється і кількості дійсних кодів не змінює.

```
 1. 95KC-8WTL-ARNJ
 2. UAXZ-JC3S-9WZS
 3. ZTKJ-NDX5-XQ5X
 4. 3FMN-N65J-KJBA
 5. QVSQ-E6WY-YJSA
 6. BAUX-CKYD-FDAD
 7. UEKT-66TR-H287
 8. WSD4-62UU-7EV7
 9. R39F-ZZKC-XYP5
10. U2NN-J685-QQ5Z
дійсних кодів: 10
перше пред'явлення коду 3: True
дійсних кодів: 9
повторне пред'явлення коду 3: False
код малими літерами без дефісів: True
вигаданий код AAAA-AAAA-AAAA: False
дійсних кодів: 8
```

4.5 Вхід у систему

Сценарій для облікового запису carol показує взаємодію всіх складників. Пароль із шести символів під час реєстрації відхилено. Повторно використаний код, три хибні паролі та хибний код разом дають п'ять невдалих спроб, після чого навіть вірні пароль і код отримують відповідь «заблоковано» і код не витрачається. Після спливання блокування вхід удається.

```
мінімальна довжина пароля: 7
реєстрація з паролем abc123: пароль коротший за 7 символів
00:00 вірний пароль, код 1            вхід дозволено  кодів: 9
01:00 вірний пароль, код 1 повторно   відмова         кодів: 9
01:10 хибний пароль, код 2            відмова         кодів: 9
01:20 хибний пароль, код 2            відмова         кодів: 9
01:30 хибний пароль, код 2            відмова         кодів: 9
01:40 вірний пароль, хибний код       відмова         кодів: 9
01:50 вірний пароль, код 2            заблоковано     кодів: 9
16:40 вірний пароль, код 2            вхід дозволено  кодів: 8
16:41 вірний пароль, код 3            вхід дозволено  кодів: 7
невідомий обліковий запис: відмова
```

4.6 Автоматизовані тести

Файл `tests/test_variant.py` містить 34 тести (таблиця 4.2). Тестовий перець задається у `tests/conftest.py`. Контрольним вектором для функції формування ключа є вектор PBKDF2-HMAC-SHA-256 з RFC 7914 [7] (пароль `passwd`, сіль `salt`, одна ітерація); для кодів відновлення RFC контрольних значень не визначає, тому перевіряються властивості, задані завданням. Запуск: `PYTHONPATH=. pytest tests/test_variant.py -v`.

Таблиця 4.2 – Групи тестів

Група	Тестів	Що перевіряється
Андерсон	3	контрольний приклад лекції, значення G і L варіанта, достатність і мінімальність L

Група	Тестів	Що перевіряється
Паролі	9	зворотність, відмова для хибного пароля, унікальність солі, формат рядка і параметри, контрольний вектор PBKDF2 з RFC 7914, залежність від перцю, пошкоджені рядки, порівняння за сталий час, порівняння з вразливим зразком
Блокування	9	параметри варіанта, 4 невдалі спроби не блокують, 5-та блокує, межа інтервалу, скидання лічильника успіхом, спроби під час блокування, повторний відлік, незалежність облікових записів, функції рівня модуля
Коди	7	10 різних кодів заданого вигляду, одноразовість, відхилення невідомого коду, нормалізація введення, зберігання лише хеш-сум
Вхід	6	пароль разом із кодом, збереження коду за хибного пароля, блокування після п'яти відмов за паролем і за кодом, невідомий обліковий запис, мінімальна довжина пароля

```
============================= test session starts ==============================
collecting ... collected 34 items

test_anderson_lecture_example PASSED
test_anderson_variant_values PASSED
test_anderson_length_is_sufficient_and_minimal PASSED
test_hash_and_verify_roundtrip PASSED
test_wrong_password_rejected PASSED
test_salt_is_unique PASSED
test_stored_format_and_parameters PASSED
test_pbkdf2_matches_rfc7914_vector PASSED
test_pepper_is_required_for_verification PASSED
test_malformed_stored_value_rejected PASSED
test_verification_uses_constant_time_comparison PASSED
test_insecure_hash_reveals_equal_passwords PASSED
test_variant_lockout_parameters PASSED
test_four_failures_do_not_lock PASSED
test_fifth_failure_locks PASSED
test_lock_expires_exactly_after_interval PASSED
test_success_resets_failure_counter PASSED
test_attempts_during_lock_do_not_extend_it PASSED
test_counter_restarts_after_lock PASSED
test_accounts_are_locked_independently PASSED
test_module_level_functions PASSED
test_ten_unique_codes_of_expected_form PASSED
test_code_sets_differ_between_calls PASSED
test_code_accepted_only_once PASSED
test_every_code_is_valid_exactly_once PASSED
test_unknown_code_rejected_and_nothing_spent PASSED
test_code_input_is_normalized PASSED
test_codes_are_not_stored_in_plaintext PASSED
test_login_requires_password_and_code PASSED
test_wrong_password_does_not_spend_code PASSED
test_login_locked_after_five_failures PASSED
test_wrong_codes_also_lock_account PASSED
test_unknown_account_denied PASSED
test_short_password_rejected_at_registration PASSED

============================== 34 passed in 5.41s ==============================
```

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

1. Залежність мінімальної довжини пароля від потужності алфавіту. Для трійки варіанта $G \geq 8{,}64 \cdot 10^{11}$. Мінімальна довжина дорівнює 9 символів для A = 26, 7 – для A = 62 і 7 – для A = 95 (таблиця 4.1, рисунок 5.1). Залежність має вигляд $L = \lceil \ln G / \ln A \rceil$: неперервна частина ln G / ln A спадає обернено пропорційно логарифму A,

тобто швидко за малих A і дедалі повільніше за великих, а заокруглення вгору перетворює її на сходинки. Через це перехід від 26 до 62 символів скорочує пароль на 2 символи, а перехід від 62 до 95 не дає нічого: точні значення 6,660 і 6,035 лежать в одному проміжку (6; 7]. Довжина 7 досягається вже за A = 51, а до 6 символів вона зменшилася б лише за A = 98. Значення 6,035 для A = 95 лише на 0,035 перевищує 6, тому сьомий символ дає великий запас: простір $95^7 \approx 6{,}98 \cdot 10^{13}$ у 80,8 раза більший за потрібний, і фактична ймовірність підбору становить $1{,}2 \cdot 10^{-8}$ замість допустимих 10^{-6}. Найбільшу ентропію серед трьох мінімальних паролів має пароль з алфавітом 95 (46,0 біта), найменшу – з алфавітом 62 (41,7 біта), бо його довжина найближча до точної межі.

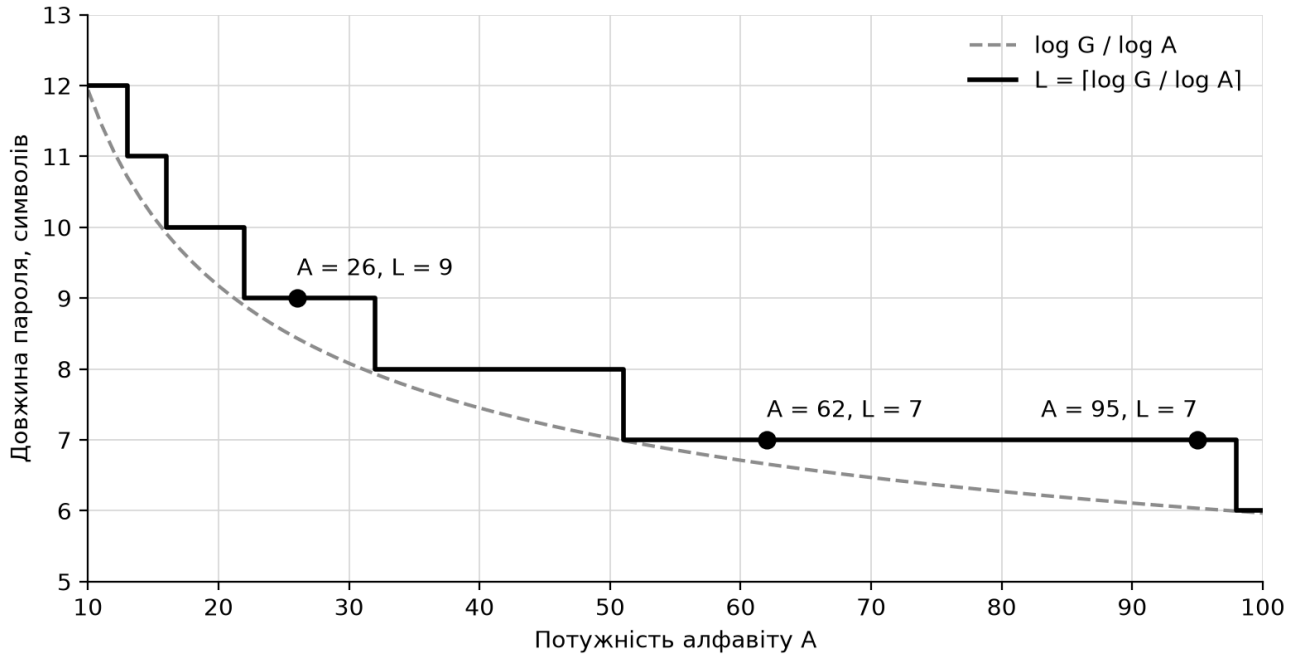


Рисунок 5.1 – Залежність мінімальної довжини пароля від потужності алфавіту

2. Час обчислення hash_password і SHA-256. На стенді одне обчислення `hash_password` із 600 000 ітерацій триває 126,7 мс (медіана із семи вимірювань), а виклик `hash_password_insecure` – 0,457 мкс (середнє з 200 000 викликів, разом із накладними витратами інтерпретатора). Перевірка стала повільнішою приблизно у 277 тис. разів: за секунду обчислюється 7,9 ключа PBKDF2 проти 2 188 184 хеш-сум SHA-256. Для користувача 127 мс на вхід непомітні, а для порушника, який викрав базу, вартість кожного кандидата зростає у стільки ж разів. Якщо перебирати з цією швидкістю половину простору 95^7 семисимвольних паролів, для SHA-256 потрібно близько 0,5 року роботи одного ядра, а для PBKDF2 – $1{,}4 \cdot 10^5$ років. Абсолютні числа залежать від обладнання (на графічних процесорах обидві функції значно швидші), але відношення зберігається, бо PBKDF2 складається з тих самих обчислень SHA-256,

повторених 1,2 млн разів. Друга вигода – повільна перевірка сама обмежує швидкість онлайн-підбору.

3. Однакові паролі, сіль і перець. `hash_password_insecure` – детермінована функція лише від пароля: обидва виклики для `password123` дали `ef92b778bafe771e…`. `hash_password` щоразу бере нові 16 байтів солі з `os.urandom`, тому два рядки для того самого пароля відрізняються і сіллю, і ключем, хоча обидва проходять перевірку. Сіль захищає від наперед обчислених (райдужних) таблиць, від розпізнавання однакових паролів у базі та від одночасного перебору всіх записів: кожен кандидат доводиться обчислювати окремо для кожної солі. Від підбору пароля одного конкретного користувача сіль не захищає, бо зберігається відкрито. Перець захищає від іншої загрози – викрадення самої бази (SQL-ін'єкція, резервна копія): без секрету, який лежить поза базою, порушник не може перевірити жодного кандидата. Це підтверджено експериментом: зі зміненим перцем перевірка вірного пароля повертає `False`. Якщо скомпрометовано і сервер застосунку, перець не допомагає, і залишається лише вартість PBKDF2.

4. Спрацювання блокування, ефективна швидкість і відмова в обслуговуванні. Блокування спрацьовує на п'ятій невдалій спробі поспіль: після чотирьох `is_allowed` повертає `True`, після п'ятої – `False` на 900 с. Межу перевірено: за секунду до спливання інтервалу спроба заборонена, рівно через 900 с – дозволена. Отже, порушник робить щонайбільше 5 спроб на 15 хв, і ефективна швидкість не перевищує 0,333 спроби/хв замість 20 – у 60 разів менше. За 30 діб це 14 400 спроб замість 864 000, і ймовірність підбору семисимвольного пароля з алфавітом 95 зменшується з $1{,}2 \cdot 10^{-8}$ до $2{,}1 \cdot 10^{-10}$. Якщо підставити ефективну швидкість у формулу, $G \geq 1{,}44 \cdot 10^{10}$, а мінімальна довжина зменшується до 8, 6 і 6 символів для алфавітів 26, 62 і 95. Скорочувати пароль на цій підставі не слід: блокування не діє на офлайн-підбір і на перебір одного пароля для багатьох облікових записів. Недолік схеми – відмова в обслуговуванні: щоб заблокувати чужий обліковий запис, досить знати його ім'я і надіслати 5 хибних паролів; повторюючи це кожні 15 хв (480 запитів на добу), порушник не дає власникові увійти взагалі. Спроби під час блокування його не подовжують, але й не заважають одразу після спливання почати новий відлік. Крім того, лічильник скидається лише успішним входом, тому п'ять помилок, зроблених за довгий час без жодного успішного входу, теж призводять до блокування. Ризик зменшують блокування за парою «обліковий запис + IP-адреса» або прогресивна затримка.

5. Порівняння кодів за сталий час. Оператор == для байтових рядків припиняє роботу на першому відмінному байті, тому час відповіді зростає з довжиною збіглого

префікса. Вимірюючи час багатьох запитів, порушник може добирати очікуване значення байт за байтом: замість перебору всього простору йому вистачає кількох сотень варіантів на кожну позицію. Для коду відновлення це означало б утрату другого чинника, для ключа пароля – відновлення збереженого значення без доступу до бази. У роботі `hmac.compare_digest` використано в обох місцях. Додатково порівнюються не самі коди, а їхні хеш-суми SHA-256 з перцем, тож навіть частковий витік через час стосувався б непередбачуваного значення, а не символів коду; цикл перевірки проходить усі збережені хеш-суми без дострокового виходу, щоб час не залежав від номера коду у списку.

ВИСНОВКИ

1. За формулою Андерсона для R = 20 спроб/хв, T = 30 діб і P = 10^{-6} отримано G ≥ 8,64 · 10^{11}; мінімальна довжина пароля – 9, 7 і 7 символів для алфавітів 26, 62 і 95, ентропія – 42,3, 41,7 і 46,0 біта. Розширення алфавіту з 62 до 95 символів довжини не зменшує. Вимогу L = 7 застосовано під час реєстрації.
2. Зберігання пароля реалізовано функцією PBKDF2-HMAC-SHA256 із 600 000 ітерацій, сіллю 16 байтів, перцем і порівнянням за сталий час; рядок має формат `pbkdf2_sha256$600000$сіль$ключ`. Реалізація збігається з контрольним вектором RFC 7914. Одне обчислення триває 126,7 мс – приблизно у 277 тис. разів довше за SHA-256 без солі.
3. Вразливий зразок дає однакову хеш-суму для однакових паролів, захищена реалізація – різні рядки; зі зміненим перцем вірний пароль не проходить перевірку.
4. Фіксоване блокування спрацьовує на п'ятій невдалій спробі та діє рівно 900 с; воно обмежує швидкість підбору значенням 0,333 спроби/хв, що в 60 разів менше за задане варіантом. Схема дозволяє сторонній особі блокувати чужий обліковий запис 480 запитами на добу.
5. Другий чинник – 10 одноразових кодів по 12 символів (60 біт); зберігаються лише їхні хеш-суми, повторне пред'явлення коду відхиляється, кількість дійсних кодів після успішного використання зменшується на одиницю.
6. Написано 34 автоматизовані тести, усі проходять.

ПЕРЕЛІК ПОСИЛАНЬ

1. Anderson R. Security Engineering: A Guide to Building Dependable Distributed Systems / R. Anderson. – 3rd ed. – Indianapolis : Wiley, 2020. – Ch. 3 (Passwords).

2. NIST SP 800-63B. Digital Identity Guidelines: Authentication and Lifecycle Management / P. Grassi [et al.]. – Gaithersburg : NIST, 2017 (with updates 2020).
3. Anderson J. P. Computer Security Technology Planning Study / J. P. Anderson. – Bedford : ESD-TR-73-51, USAF Electronic Systems Division, 1972. – Vol. II.
4. OWASP Cheat Sheet Series. Password Storage Cheat Sheet [Електронний ресурс]. – Режим доступу: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
5. RFC 8018. PKCS #5: Password-Based Cryptography Specification Version 2.1 / K. Moriarty, B. Kaliski, A. Rusch. – IETF, 2017.
6. Python 3 Documentation. hmac – Keyed-Hashing for Message Authentication [Електронний ресурс]. – Режим доступу: https://docs.python.org/3/library/hmac.html
7. RFC 7914. The scrypt Password-Based Key Derivation Function / C. Percival, S. Josefsson. – IETF, 2016. – Section 11 (Test Vectors for PBKDF2 with HMAC-SHA-256).
8. Соколовський В. В. Навчально-методичний комплекс дисципліни «Безпека програмного забезпечення»: код лабораторних робіт [Електронний ресурс]. – Режим доступу: https://github.com/sokolusha/software-security-course

ДОДАТОК А. ПРОГРАМНИЙ КОД

Повний програмний код роботи розміщено в репозиторії: https://github.com/kqjqxb/Software-Security, коміт 123d71c. Файли роботи – у каталозі `lab3/lab03_auth`: `auth/passwords.py`, `auth/lockout.py`, `auth/recovery.py`, `auth/service.py`, `demo.py`, `tests/conftest.py`, `tests/test_variant.py`.